

软件工程学期实训 II

OpenSpec 组员提示词使用指南

home-camera-monitor | 2026-07-08

仓库: <https://github.com/niu-spec/home-camera-monitor>

在线文档: <https://github.com/niu-spec/home-camera-monitor/blob/dev/docs/OpenSpec组员上手指南.md>

核心理解

OpenSpec 不是 Cursor 专属工具, 而是仓库里的 Markdown 规格文档。组员只需 git pull 拿到 openspec/ 目录, 按 tasks.md 写代码, 再 push 回 GitHub。任意 AI Agent (ChatGPT、Claude、Copilot 等) 都能用。

一、三步上手

1. git pull 拉最新 dev 代码, 切到自己的 feature 分支
2. 打开 openspec/changes/ai-video-integration/tasks.md, 找到自己那节任务
3. 把规格文件内容贴给 Agent, 复制下方提示词, 说「按 tasks X.X 实现」
4. 做完勾选 tasks.md, push 到 feature 分支, PR 到 dev, Assign 牛雨昊 Review

二、要先读的文件

负责人	先看这些文件
王梓铭 (C)	tasks.md 第 2 节 + design.md + specs/
李东礼 (D)	tasks.md 第 3 节 + design.md + specs/
刘帅华 (E)	openspec/specs/ 下对应 capability
苏哲勋 (B)	openspec/specs/video-stream/spec.md
刘澎潮 (F)	openspec/specs/ 各 capability

三、Git 命令

```
git checkout dev
git pull origin dev
git checkout feature/face # 王梓铭
# git checkout feature/detection # 李东礼
# git checkout feature/business # 刘帅华

git add .
git commit -m "feat(face): 完成 tasks 2.1-2.2"
git push origin feature/face
```

四、PR 描述模板

关联 OpenSpec: ai-video-integration
完成任务: 2.1, 2.2
规格参考: openspec/changes/ai-video-integration/specs/

测试说明:

- 本地接口可访问
- 符合 spec 中的 Scenario

五、通用开场提示词（所有人第一次对话先发）

我在做 home-camera-monitor 项目（居家智能摄像头，Django + Vue3）。

请先阅读我提供的以下文件，理解项目约定和任务要求，不要急着写代码：

1. openspec/config.yaml
2. openspec/changes/ai-video-integration/tasks.md
3. openspec/changes/ai-video-integration/design.md
4. openspec/changes/ai-video-integration/specs/ai-video-integration/spec.md

读完后告诉我：你理解了我的哪些任务、涉及哪些文件、接口约定是什么。

六、做完后发给 Agent 的提示词

任务 [X.X] 已完成，请帮我：

1. 检查是否符合 openspec/changes/ai-video-integration/specs/ 里的 Scenario
2. 列出需要勾选 tasks.md 的哪几项（把 [] 改成 [x]）
3. 建议的 git commit message

七、各角色专用提示词

苏哲勋 (B) — 流媒体/MediaMTX/视频拉流

分支: feature/nginx 或 feature/django-video-stream

- openspec/specs/video-stream/spec.md
- openspec/changes/ai-video-integration/design.md

修改 process_frame() (视频流)

我要修改 backend/apps/video_stream/services.py 的 process_frame()。

请先读:

- openspec/specs/video-stream/spec.md
- openspec/changes/ai-video-integration/design.md

要求保留 face 和 detection 的接入扩展点, 不要破坏处理器链结构。

流程: face 先处理 → detection 再处理 → 返回标注后的帧。

王梓铭 (C) — AI人脸/人员统计

分支: feature/face

- openspec/config.yaml
- openspec/changes/ai-video-integration/tasks.md (第2节)
- openspec/changes/ai-video-integration/design.md
- openspec/changes/ai-video-integration/specs/ai-video-integration/spec.md

任务 2.1 — 人脸检测与编码

按 openspec/changes/ai-video-integration/tasks.md 的 2.1, 在 backend/apps/face/services.py 中接入 dlib 人脸检测与 128 维编码比对。

要求:

- 使用 face_recognition 库 (HOG 模型)
- 编码维度必须是 128
- 与 registered_faces.json 家人库比对
- 不要改动无关文件

任务 2.2 — 注册 API

完成任务 2.2: 实现 POST /api/face/register/ 与家人库持久化。

参考现有 backend/apps/face/views.py 和 services.py, 注册成功后同时写入数据库 FamilyMember 和 registered_faces.json。

任务 2.3 + 2.4 — presence 与陌生人告警

完成任务 2.3 和 2.4:

- process_frame() 处理后更新 GET /api/home/presence/ 的人数统计
- 检测到陌生人时触发 FACE_UNKNOWN 告警 (调用 apps.alerts.services.create_alert)
- 告警有 30 秒冷却, 避免重复触发

接入视频流 (最关键)

按 openspec/changes/ai-video-integration/design.md 的处理器链设计, 把 backend/apps/face/services.py 接入 backend/apps/video_stream/services.py 的 process_frame()。

流程: face 先处理 → 画框标注 → 更新 presence → 返回标注后的帧。
现有 face 模块代码已有, 重点是挂到视频流钩子上。

李东礼 (D) — AI危险区域/异常检测

分支: feature/detection

- openspec/config.yaml
- openspec/changes/ai-video-integration/tasks.md (第 3 节)
- openspec/changes/ai-video-integration/design.md
- openspec/changes/ai-video-integration/specs/ai-video-integration/spec.md

任务 3.1 — HOG 行人检测

按 tasks.md 3.1, 在 backend/apps/detection/services.py 中用 HOG 行人检测判断人员是否进入危险区域。person_boxes 为 None 时内部自动跑 HOG。

任务 3.2 — 厨房禁区闯入

完成任务 3.2: 当 child 角色的人进入厨房禁区多边形时, 触发 INTRUSION 告警。区域配置从 apps.zones.models.Zone 读取, forbidden_roles 包含 'child' 的区域才检测。

任务 3.3 — 积水/着火/跌倒

完成任务 3.3: 实现积水/着火/跌倒检测, 告警类型分别为 WATER、FIRE、FALL。参考 detection/services.py 现有 HSV 检测逻辑, 确保调用 create_alert 写入数据库。

接入视频流

按 design.md, 在 process_frame() 处理器链中, face 处理完之后接 detection:

- 传入 stream_id、zones、person_boxes、face_roles
- 用 draw_overlays() 在帧上画检测框和区域多边形
- 告警类型用 INTRUSION / WATER / FIRE / FALL

刘帅华 (E) — Django业务/数据库/Swagger

分支: feature/business

- openspec/config.yaml
- openspec/specs/alerts-events/spec.md (或对应 capability)

修改现有 API

我要修改 [告警/区域/家庭] 相关 API。

请先读 openspec/specs/alerts-events/spec.md (或对应 spec) ,
确保改动符合规格里的 Scenario, 不要破坏现有接口格式。

新增功能

我要新增 [功能名] API。

请参考 openspec/changes/archive/ 里已有 change 的格式,
帮我起草 proposal.md 和 tasks.md, 我先给组长确认再写代码。

刘澎湖 (F) — 专职文档

分支: — (不写代码)

- openspec/specs/ 下各 capability
- openspec/changes/archive/ 已归档 change

文档对齐规格

我在写项目文档, 请以 openspec/specs/ 为权威来源,
帮我检查 [某功能] 的描述是否与 spec 一致。
如发现 spec 与实现不一致, 请列出差异点。

八、常见问题

Q: 没有 Cursor 能用吗?

A: 可以。OpenSpec 文件在 Git 仓库里, 任何编辑器 + Agent 都能用。

Q: tasks 全部做完后?

A: 在 PR 或群里通知牛雨昊, 由组长归档 change。

Q: spec 和代码不一致?

A: 在群里说明, 不要擅自改 openspec/specs/ 主目录。

Q: 要做的新功能没有 change?

A: 联系牛雨昊新建, 或参考 archive/ 范例起草。

一句话: git pull → 读 tasks.md → 贴提示词给 Agent → 勾选 tasks → PR 到 dev